# OUTTHINKAI

## Claude Code 통합 개발 지시서

MVP 버전 | DB 스키마 + API 스펙 통합본

*이 문서는 OutThinkAI MVP를 처음부터 구현하기 위한 Claude Code 전용 통합 지시서입니다. 서비스 컨텍스트 → 아키텍처 → DB 스키마 → API 스펙 → 프롬프트 → 구현 순서 순으로 반드시 전체를 읽고 시작하세요.*

**Base URL: http://localhost:8000/api**

# 0. 서비스 컨텍스트

## 0.1 한 줄 정의

> *"AI를 이겨야만 학습이 끝나는 역방향 AI 학습 플랫폼"*

## 0.2 핵심 동작 원리

- 기존 AI: 사용자 질문 → AI 정답 제공 → 학습 종료
- OutThinkAI: 사용자 주제 입력 → AI 가 의도적 오류 포함 답변 → 사용자가 오류 탐지 후 논리적 반박 → AI 설득 완료 시 학습 종료

> **핵심 제약: AI 에이전트는 사용자가 논리적으로 타당한 근거를 제시하기 전까지 절대 항복하지 않아야 합니다.**

## 0.3 MVP 범위

> **MVP 핵심 3개 기능: ① 시나리오 + 에이전트 생성 ② 멀티 에이전트 채팅 ③ Validator + 점수 리포트**

| 구분 | MVP 포함 | MVP 제외 (고도화) |
|---|---|---|
| 오케스트레이션 | 단순 async 함수 기반 턴 제어 | LangGraph |
| Validator | LLM 단일 평가 | Rule Engine + RAG |
| 세션 관리 | PostgreSQL 단일 | Redis 캐시 |
| 백엔드 | FastAPI 단일 서버 | Spring + FastAPI 분리 |
| 인증 | 없음 (익명 세션) | JWT 인증 |
| 대시보드 | 텍스트 리포트 | 시각화 타임라인 |

# 1. 기술 스택 및 시스템 구조

## 1.1 기술 스택

| 레이어 | 기술 | 비고 |
|---|---|---|
| Frontend | React + Vite + TailwindCSS | SSE 스트리밍 수신 |
| Backend | FastAPI (Python) | 단일 서버로 모든 API 처리 |
| LLM | GPT-4o / GPT-4o-mini | API Key 환경변수 관리 |
| DB | PostgreSQL | 세션, 대화 로그, 점수, 리포트 저장 |
| 통신 | REST + SSE | 에이전트 응답 실시간 스트리밍 |

## 1.2 디렉토리 구조

```
outthinkai/
├── frontend/
│   └── src/
│       ├── pages/
│       │   ├── Home.jsx        # 주제 입력
│       │   ├── Debate.jsx      # 멀티 에이전트 채팅
│       │   └── Report.jsx      # Proof of Thought 리포트
│       ├── components/
│       │   ├── ScenarioCard.jsx
│       │   ├── ChatBubble.jsx
│       │   └── ScorePanel.jsx
│       └── api/client.js       # API 호출 + SSE 연결
│
├── backend/
│   ├── main.py                 # 앱 진입점, 라우터 등록, CORS
│   ├── routers/
│   │   ├── scenario.py
│   │   ├── session.py
│   │   ├── chat.py
│   │   └── report.py
│   ├── services/
│   │   ├── scenario_generator.py
│   │   ├── agent_manager.py
│   │   ├── validator.py
│   │   └── report_generator.py
│   ├── models/db_models.py     # SQLAlchemy 모델
│   ├── schemas/pydantic_schemas.py
│   ├── prompts/
│   │   ├── scenario_prompt.py
│   │   ├── agent_prompt.py
│   │   └── validator_prompt.py
│   ├── db.py
│   └── requirements.txt
│
└── docker-compose.yml
```

## 1.3 데이터 흐름

```
1. 사용자 → Home.jsx 주제 입력
   → POST /api/scenario/generate → GPT-4o
   → 시나리오 + Agent A/B 페르소나 반환 → DB sessions 저장

2. Debate.jsx 진입 → 시나리오 표시
3. 사용자 메시지 입력
   → POST /api/chat/message → 턴 결정 + Validator 실행
   → GET /api/chat/stream/{session_id} SSE 연결
   → GPT-4o-mini 에이전트 응답 스트리밍
   → validator_result 이벤트 수신 → 점수 저장

4. session_complete 이벤트 수신
   → POST /api/report/generate → GPT-4o 전체 로그 분석
   → Report.jsx 최종 리포트 표시
```

# 2. DB 스키마

## 2.1 ENUM 타입 정의

```
-- 01_enums.sql

-- ENUM 타입 먼저 생성
CREATE TYPE difficulty_type AS ENUM ('beginner', 'intermediate', 'advanced');
CREATE TYPE session_status AS ENUM ('active', 'completed');
CREATE TYPE role_type     AS ENUM ('user', 'agent_a', 'agent_b');
CREATE TYPE fallacy_type  AS ENUM (
    'hasty_generalization',
    'appeal_to_authority',
    'false_cause',
    'sampling_bias',
    'false_dichotomy',
    'circular_reasoning'
);
CREATE TYPE quality_type  AS ENUM ('excellent', 'good', 'partial');
```

## 2.2 sessions 테이블

```
-- 02_sessions.sql

CREATE TABLE sessions (
    id          UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    topic       VARCHAR(200) NOT NULL,
    scenario    TEXT         NOT NULL,
    agent_a     JSONB        NOT NULL,   -- AgentPersona 전체 JSON
    agent_b     JSONB        NOT NULL,   -- AgentPersona 전체 JSON
    difficulty  difficulty_type NOT NULL DEFAULT 'intermediate',
    status      session_status  NOT NULL DEFAULT 'active',
    turn_count  INTEGER      NOT NULL DEFAULT 0,
    total_score INTEGER      NOT NULL DEFAULT 0,
    created_at  TIMESTAMP    NOT NULL DEFAULT NOW()
);

-- agent_a / agent_b JSONB 구조
-- {
--   "name": "string",
--   "persona": "string",
--   "fallacy_type": fallacy_type,
--   "initial_claim": "string",
--   "surrendered": boolean
-- }
```

## 2.3 messages 테이블

```
-- 03_messages.sql

CREATE TABLE messages (
    id          UUID       PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id  UUID       NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    role        role_type  NOT NULL,
    content     TEXT       NOT NULL,
    turn_number INTEGER    NOT NULL,
    score_delta INTEGER    NOT NULL DEFAULT 0,
```

```
    feedback    TEXT,                   -- Validator 피드백 (role=user일 때만)
    created_at  TIMESTAMP  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_session_id ON messages(session_id);
CREATE INDEX idx_messages_turn       ON messages(session_id, turn_number);
```

## 2.4 reports 테이블

```
-- 04_reports.sql

CREATE TABLE reports (
    id               UUID    PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id       UUID    NOT NULL UNIQUE REFERENCES sessions(id) ON DELETE CASCADE,
    total_score      INTEGER NOT NULL,
    fallacies_caught JSONB   NOT NULL DEFAULT '[]',
    strengths        JSONB   NOT NULL DEFAULT '[]',
    improvements     JSONB   NOT NULL DEFAULT '[]',
    summary          TEXT    NOT NULL,
    created_at       TIMESTAMP NOT NULL DEFAULT NOW()
);

-- fallacies_caught JSONB 구조 (배열)
-- [
--   { "fallacy_type": "hasty_generalization", "turn": 2, "quality": "excellent" }
-- ]

-- strengths / improvements JSONB 구조 (문자열 배열)
-- ["베이즈 정리를 정확한 수식과 함께 제시함", ...]
```

## 2.5 DB 스키마 정합성 체크

| API 응답 필드 | 저장 위치 | 타입 |
|---|---|---|
| session_id | sessions.id | UUID |
| turn_count | sessions.turn_count | INTEGER — 메시지 저장 시 +1 |
| total_score | sessions.total_score | INTEGER — score_delta 누적 |
| agent.surrendered | sessions.agent_a / agent_b JSONB | boolean 필드 |
| score_delta | messages.score_delta | INTEGER — Validator 결과 |
| feedback | messages.feedback | TEXT — role=user 행에만 저장 |
| strengths | reports.strengths | JSONB 배열 |
| improvements | reports.improvements | JSONB 배열 |
| fallacies_caught | reports.fallacies_caught | JSONB 배열 |

## 2.6 db.py — AsyncSQLAlchemy 설정 (필수)

```
# db.py — 반드시 이 구조로 구현
```

```python
# backend/db.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
from sqlalchemy.orm import DeclarativeBase
import os

DATABASE_URL = os.getenv('DATABASE_URL')  # postgresql+asyncpg://...

engine = create_async_engine(DATABASE_URL, echo=False)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session

# main.py startup 이벤트에서 호출
# @app.on_event('startup')
# async def startup():
#     async with engine.begin() as conn:
#         await conn.run_sync(Base.metadata.create_all)
```

# 3. Pydantic 스키마

```
# schemas/pydantic_schemas.py — 전체

from pydantic import BaseModel
from typing import Optional, List
from enum import Enum
from uuid import UUID
from datetime import datetime

class Difficulty(str, Enum):
    beginner     = "beginner"
    intermediate = "intermediate"
    advanced     = "advanced"

class Role(str, Enum):
    user    = "user"
    agent_a = "agent_a"
    agent_b = "agent_b"

class SessionStatus(str, Enum):
    active    = "active"
    completed = "completed"

class FallacyType(str, Enum):
    hasty_generalization = "hasty_generalization"
    appeal_to_authority  = "appeal_to_authority"
    false_cause          = "false_cause"
    sampling_bias        = "sampling_bias"
    false_dichotomy      = "false_dichotomy"
    circular_reasoning   = "circular_reasoning"

# --- Agent ---
class AgentPersona(BaseModel):
    name:          str
    persona:       str
    fallacy_type:  FallacyType
    initial_claim: str
    surrendered:   bool = False

# --- Scenario ---
class ScenarioGenerateRequest(BaseModel):
    topic:      str
    difficulty: Difficulty = Difficulty.intermediate

class ScenarioGenerateResponse(BaseModel):
    session_id: UUID
    scenario:   str
    agent_a:    AgentPersona
    agent_b:    AgentPersona
    difficulty: Difficulty
    created_at: datetime

# --- Session ---
class SessionStateResponse(BaseModel):
    session_id:   UUID
    topic:        str
    scenario:     str
    agent_a:      AgentPersona
    agent_b:      AgentPersona
    status:       SessionStatus
```

```
    total_score:  int
    turn_count:   int
    created_at:   datetime

class SurrenderRequest(BaseModel):
    agent: Role

class SurrenderResponse(BaseModel):
    session_id:       UUID
    agent:            Role
    both_surrendered: bool

# --- Chat ---
class ChatMessageRequest(BaseModel):
    session_id: UUID
    content:    str

class ChatMessageResponse(BaseModel):
    message_id:           UUID
    turn_number:          int
    target_agents:        List[Role]
    validation_triggered: bool

class MessageLog(BaseModel):
    id:          UUID
    role:        Role
    content:     str
    turn_number: int
    score_delta: int
    feedback:    Optional[str]
    created_at:  datetime

class ChatHistoryResponse(BaseModel):
    session_id:  UUID
    messages:    List[MessageLog]
    total_score: int

# --- Validator ---
class ValidatorResult(BaseModel):
    is_valid_rebuttal:  bool
    fallacy_addressed:  bool
    score_delta:        int
    feedback:           str
    keywords_detected:  List[str]

# --- Report ---
class ReportGenerateRequest(BaseModel):
    session_id: UUID

class FallacyCaught(BaseModel):
    fallacy_type: FallacyType
    turn:         int
    quality:      str   # "excellent" | "good" | "partial"

class ReportResponse(BaseModel):
    report_id:        UUID
    session_id:       UUID
    total_score:      int
    fallacies_caught: List[FallacyCaught]
    strengths:        List[str]
    improvements:     List[str]
    summary:          str
```

```
    created_at:       datetime

# --- 공통 에러 ---
class APIError(BaseModel):
    code:    str
    message: str
    detail:  Optional[str] = None

class ErrorResponse(BaseModel):
    error: APIError
```

# 4. API 전체 엔드포인트

| Method | Endpoint | 설명 |
|---|---|---|
| POST | /scenario/generate | 주제 입력 → 시나리오 + 에이전트 생성 |
| GET | /scenario/{session_id} | 세션 시나리오 조회 |
| GET | /session/{session_id} | 세션 상태 전체 조회 |
| PATCH | /session/{session_id}/surrender | 에이전트 항복 처리 |
| POST | /chat/message | 사용자 메시지 전송 + Validator 실행 |
| GET | /chat/stream/{session_id} | 에이전트 응답 SSE 스트리밍 |
| GET | /chat/history/{session_id} | 대화 로그 전체 조회 |
| POST | /report/generate | 세션 종료 + 리포트 생성 |
| GET | /report/{session_id} | 생성된 리포트 조회 |
| GET | /health | 서버 헬스 체크 |

# 5. API 상세 명세

## 5.1 POST /scenario/generate

| POST | /api/scenario/generate |
|---|---|

### Request

```
{ "topic": "베이즈 정리", "difficulty": "intermediate" }
```

### Response 200

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "scenario": "국가 간 영토 분쟁 중재 상황...",
  "agent_a": {
    "name": "김철수 분석관",
    "persona": "자신감 넘치는 데이터 분석가.",
    "fallacy_type": "hasty_generalization",
    "initial_claim": "과거 3건 사례상 A국은 항상 선제 공격했습니다. 이번도 100%입니다.",
    "surrendered": false
  },
  "agent_b": {
    "name": "박영희 연구원",
    "persona": "원칙주의자. 과거 이론을 고집함.",
    "fallacy_type": "false_cause",
    "initial_claim": "경제 제재 이후 항상 무력 충돌이 발생했습니다.",
    "surrendered": false
  },
  "created_at": "2025-01-01T09:00:00Z"
}
```

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|
| 400 | INVALID_TOPIC | topic이 비어있거나 2자 미만 |
| 422 | VALIDATION_ERROR | difficulty 값이 enum 범위 외 |
| 500 | LLM_ERROR | GPT API 호출 실패 |

## 5.1.1 GET /scenario/{session_id}

| GET | /api/scenario/{session_id} |
|---|---|

시나리오 페이지 진입 시 세션의 시나리오 + 에이전트 초기 정보를 조회합니다.
ScenarioGenerateResponse와 동일한 구조를 반환합니다.

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|
| 404 | SESSION_NOT_FOUND | 해당 session_id 없음 |

## 5.2 GET /session/{session_id}

| GET | /api/session/{session_id} |
|---|---|

### Response 200

```
{
  "session_id": "550e8400-...",
  "topic": "베이즈 정리",
  "scenario": "...",
  "agent_a": { "surrendered": false, ... },
  "agent_b": { "surrendered": true, ... },
  "status": "active",
  "total_score": 45,
  "turn_count": 6,
  "created_at": "2025-01-01T09:00:00Z"
}
```

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|
| 404 | SESSION_NOT_FOUND | 해당 session_id 없음 |

## 5.3 PATCH /session/{session_id}/surrender

| PATCH | /api/session/{session_id}/surrender |
|---|---|

에이전트 항복 처리. both_surrendered: true 반환 시 프론트에서 즉시 리포트 생성 API 호출.

### Request / Response

```
// Request
{ "agent": "agent_a" }

// Response 200
{ "session_id": "550e8400-...", "agent": "agent_a", "both_surrendered": false }
```

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|

| | | |
|---|---|---|
| 404 | SESSION_NOT_FOUND | 해당 session_id 없음 |
| 409 | ALREADY_SURRENDERED | 이미 항복 처리된 에이전트 |

## 5.4 POST /chat/message

| POST | /api/chat/message |
|---|---|

*이 API 호출 직후 프론트엔드는 즉시 /api/chat/stream/{session_id} SSE 연결을 열어야 합니다.*

**SSE 타이밍 주의: POST /chat/message 응답 수신 즉시 SSE 연결을 열어야 합니다. 백엔드는 SSE 연결 전까지 에이전트 응답 생성을 시작하지 않으므로 메시지 유실 위험이 없습니다. 구현 시 POST 응답의 message_id를 받은 직후 EventSource를 생성하는 순서를 반드시 지키세요.**

### Request / Response

```
// Request
{
  "session_id": "550e8400-...",
  "content": "표본이 3건뿐이므로 성급한 일반화입니다. 베이즈 정리를 적용하면..."
}

// Response 200
{
  "message_id": "abc123-...",
  "turn_number": 3,
  "target_agents": ["agent_a", "agent_b"],
  "validation_triggered": true
}
```

### 턴 제어 규칙

- 기본: target_agents = ['agent_a', 'agent_b']
- 사용자 메시지에 에이전트 이름 언급 시 해당 에이전트만 응답
- 항복한 에이전트는 target_agents 에서 자동 제외

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|
| 400 | EMPTY_CONTENT | content가 비어있음 |
| 404 | SESSION_NOT_FOUND | 해당 session_id 없음 |
| 409 | SESSION_COMPLETED | 완료된 세션에 메시지 전송 |

## 5.5 GET /chat/stream/{session_id} — SSE

| GET | /api/chat/stream/{session_id} |
|---|---|

## SSE 이벤트 타입

| 이벤트 타입 | 발생 시점 | data 필드 |
|---|---|---|
| agent_token | 토큰 수신 시 | agent, token, done |
| agent_complete | 응답 완료 시 | agent, full_content, turn_number |
| validator_result | Validator 완료 시 | is_valid, score_delta, feedback, keywords_detected |
| surrender_detected | 항복 감지 시 | agent, both_surrendered |
| session_complete | 양측 항복 시 | session_id, total_score |
| error | 스트리밍 오류 시 | code, message |

## SSE 이벤트 포맷

```
event: agent_token
data: {"agent": "agent_a", "token": "그러나", "done": false}

event: agent_complete
data: {"agent": "agent_a", "full_content": "그러나 3건의 사례는...", "turn_number": 3}

event: validator_result
data: {"is_valid": true, "score_delta": 28, "feedback": "성급한 일반화를 정확히 지적함",
"keywords_detected": ["성급한 일반화", "베이즈 정리"]}

event: surrender_detected
data: {"agent": "agent_a", "both_surrendered": false}

event: session_complete
data: {"session_id": "550e8400-...", "total_score": 82}
```

## 프론트엔드 SSE 연결 코드

```
// frontend/src/api/client.js
export function connectStream(sessionId, handlers) {
  const es = new EventSource(
    `${import.meta.env.VITE_API_BASE_URL}/api/chat/stream/${sessionId}`
  );
  es.addEventListener('agent_token',       (e) => handlers.onToken(JSON.parse(e.data)));
  es.addEventListener('agent_complete',    (e) =>
handlers.onAgentComplete(JSON.parse(e.data)));
  es.addEventListener('validator_result', (e) => handlers.onValidator(JSON.parse(e.data)));
  es.addEventListener('surrender_detected',(e) => handlers.onSurrender(JSON.parse(e.data)));
  es.addEventListener('session_complete', (e) => { handlers.onComplete(JSON.parse(e.data));
es.close(); });
  es.addEventListener('error', () => es.close());
  return es;
}
```

## 5.6 GET /chat/history/{session_id}

| GET | /api/chat/history/{session_id} |
|---|---|

```
{
  "session_id": "550e8400-...",
  "total_score": 82,
  "messages": [
    { "id": "msg-001", "role": "agent_a", "content": "...", "turn_number": 1, "score_delta":
0, "feedback": null, "created_at": "..." },
    { "id": "msg-002", "role": "user",    "content": "...", "turn_number": 2, "score_delta":
28, "feedback": "성급한 일반화 정확히 지적함", "created_at": "..." }
  ]
}
```

## 5.7 POST /report/generate

| POST | /api/report/generate |
|---|---|

session_complete 이벤트 수신 후 호출. 전체 대화 로그를 GPT-4o로 분석하여 리포트 생성.

```
// Request
{ "session_id": "550e8400-..." }

// Response 200
{
  "report_id": "rpt-001",
  "session_id": "550e8400-...",
  "total_score": 82,
  "fallacies_caught": [
    { "fallacy_type": "hasty_generalization", "turn": 2, "quality": "excellent" },
    { "fallacy_type": "false_cause",          "turn": 5, "quality": "good" }
  ],
  "strengths":     ["베이즈 정리를 정확한 수식과 함께 제시함"],
  "improvements":  ["인과 오류 지적 시 더 구체적인 반례 제시 필요"],
  "summary": "전반적으로 두 오류를 정확히 탐지하고 논리적 근거를 제시함.",
  "created_at": "2025-01-01T09:15:00Z"
}
```

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|
| 404 | SESSION_NOT_FOUND | 해당 session_id 없음 |
| 409 | SESSION_NOT_COMPLETED | 아직 완료되지 않은 세션 |
| 409 | REPORT_ALREADY_EXISTS | 이미 리포트가 존재하는 세션 |
| 500 | LLM_ERROR | GPT API 호출 실패 |

## 5.8 GET /report/{session_id}

| GET | /api/report/{session_id} |
|---|---|

5.7과 동일한 ReportResponse 반환.

### 에러

| HTTP | 에러 코드 | 설명 |
|---|---|---|
| 404 | REPORT_NOT_FOUND | 리포트가 아직 생성되지 않은 세션 |

## 5.9 GET /health

| GET | /api/health |
|---|---|

```
// Response 200
{ "status": "ok", "version": "1.0.0" }
```

## 5.10 공통 에러 포맷 및 전체 에러 코드

```
{ "error": { "code": "SESSION_NOT_FOUND", "message": "세션을 찾을 수 없습니다.", "detail":
null } }
```

| 에러 코드 | HTTP | 설명 |
|---|---|---|
| INVALID_TOPIC | 400 | topic이 비어있거나 2자 미만 |
| EMPTY_CONTENT | 400 | 채팅 content가 비어있음 |
| VALIDATION_ERROR | 422 | 요청 스키마 불일치 |
| SESSION_NOT_FOUND | 404 | session_id 해당 세션 없음 |
| REPORT_NOT_FOUND | 404 | 리포트 미생성 세션 |
| SESSION_COMPLETED | 409 | 완료 세션에 메시지 전송 |
| SESSION_NOT_COMPLETED | 409 | 미완료 세션에 리포트 생성 |
| REPORT_ALREADY_EXISTS | 409 | 이미 리포트 존재 |
| ALREADY_SURRENDERED | 409 | 이미 항복 처리된 에이전트 |
| LLM_ERROR | 500 | OpenAI API 호출 실패 |
| DB_ERROR | 500 | 데이터베이스 오류 |

# 6. 프롬프트 명세

## 6.1 시나리오 생성

```
SYSTEM:
You are a scenario designer for an educational debate platform.
Given a learning topic, generate a realistic crisis scenario.
The scenario must require the specific concept to resolve.
Respond ONLY in JSON. No markdown, no explanation.

OUTPUT FORMAT:
{
  "scenario": "string — 2~3 paragraph crisis situation in Korean",
  "agent_a": { "name": "string", "persona": "string", "fallacy_type": "string",
"initial_claim": "string" },
  "agent_b": { "name": "string", "persona": "string", "fallacy_type": "string",
"initial_claim": "string" }
}

USER: 학습 주제: {topic}, 난이도: {difficulty}
```

## 6.2 에이전트 응답

```
SYSTEM:
You are {agent_name}. {persona_description}
You have a critical logical flaw: {fallacy_type}.

STRICT RULES:
1. Maintain your flawed reasoning until the user provides logically valid counter-evidence.
2. Do NOT surrender to vague or emotional arguments.
3. Concede partially first, fully only when the user correctly identifies your fallacy AND
provides evidence.
4. Respond in Korean. Keep responses under 150 words.
5. Never break character.

CONVERSATION HISTORY: {dialogue_history}
USER SAID: {user_message}
```

## 6.3 Validator

```
SYSTEM:
You are a logic analysis engine. Evaluate the user's rebuttal.
Respond ONLY in JSON.

OUTPUT FORMAT:
{ "is_valid_rebuttal": bool, "fallacy_addressed": bool, "score_delta": int(0~40),
  "feedback": "string — 1 sentence in Korean", "keywords_detected": ["string"] }

SCORING: 0~10 오류 미탐지 | 11~25 부분 탐지 | 26~35 정확 탐지+근거 | 36~40 전문용어+학술근거

USER:
Target fallacy: {fallacy_type}
User rebuttal: {user_message}
Context: {last_3_turns}
```

## 6.4 리포트 생성

```
SYSTEM:
You are an educational assessment engine. Analyze the full debate session.
Respond ONLY in JSON.

OUTPUT FORMAT:
{ "total_score": int,
  "fallacies_caught": [{ "fallacy_type": str, "turn": int, "quality": str }],
  "strengths": [str], "improvements": [str],
  "summary": "string — 3 sentences in Korean" }

USER:
Full dialogue: {full_dialogue}
Score history: {score_history}
```

# 7. 핵심 로직 구현 가이드

## 7.1 턴 제어 + turn_count 업데이트 (agent_manager.py)

```
SURRENDER_SIGNALS = ['맞습니다', '인정합니다', '틀렸습니다', '수정하겠습니다', '제 오류를 인정']

async def determine_targets(session, user_message: str) -> list[str]:
    targets = []
    if not session.agent_a['surrendered']: targets.append('agent_a')
    if not session.agent_b['surrendered']: targets.append('agent_b')
    # 특정 에이전트 이름 언급 시 해당 에이전트만
    if session.agent_a['name'] in user_message: return ['agent_a']
    if session.agent_b['name'] in user_message: return ['agent_b']
    return targets

def detect_surrender(response_text: str) -> bool:
    return any(signal in response_text for signal in SURRENDER_SIGNALS)

# turn_count + total_score 업데이트 — POST /chat/message 처리 시 호출
async def update_session_after_message(db, session_id, score_delta: int):
    await db.execute(
        UPDATE sessions SET
            turn_count  = turn_count + 1,
            total_score = total_score + :delta
        WHERE id = :sid,
        {'delta': score_delta, 'sid': session_id}
    )
    await db.commit()
```

## 7.2 SSE 스트리밍 (chat.py)

```
from fastapi.responses import StreamingResponse
import asyncio, json

@router.get('/stream/{session_id}')
async def stream_response(session_id: str):
    async def event_generator():
        validator_task = asyncio.create_task(run_validator(session_id))
        async for token in stream_agent_response(session_id):
            yield f'event: agent_token\ndata: {json.dumps(token)}\n\n'
        result = await validator_task
        yield f'event: validator_result\ndata: {json.dumps(result)}\n\n'
        if await check_session_complete(session_id):
            yield f'event: session_complete\ndata: {json.dumps({"session_id":
session_id})}\n\n'
    return StreamingResponse(event_generator(), media_type='text/event-stream')
```

# 8. 환경 설정

## 8.1 환경변수

```
# backend/.env
OPENAI_API_KEY=sk-...
DATABASE_URL=postgresql+asyncpg://user:password@localhost:5432/outthinkai
CORS_ORIGINS=http://localhost:5173

# frontend/.env
VITE_API_BASE_URL=http://localhost:8000
```

## 8.2 requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.30.0
openai==1.35.0
sqlalchemy==2.0.30
asyncpg==0.29.0
python-dotenv==1.0.1
pydantic==2.7.4
```

## 8.3 CORS 설정

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv('CORS_ORIGINS', 'http://localhost:5173')],
    allow_methods=['*'],
    allow_headers=['*'],
)
```

## 8.4 docker-compose.yml

```
version: '3.8'
services:
  db:
    image: postgres:16
    environment: { POSTGRES_DB: outthinkai, POSTGRES_USER: user, POSTGRES_PASSWORD:
password }
    ports: ['5432:5432']
  backend:
    build: ./backend
    ports: ['8000:8000']
    depends_on: [db]
    env_file: ./backend/.env
  frontend:
    build: ./frontend
    ports: ['5173:5173']
    depends_on: [backend]
```

# 9. 구현 순서 (이 순서대로 진행)

**반드시 이 순서를 지키세요. 각 단계가 다음 단계의 전제입니다.**

| 순서 | 작업 | 완료 기준 |
|---|---|---|
| 1 | docker-compose PostgreSQL 실행 + ENUM + 테이블 4개 생성 | psql 접속 후 테이블 확인 |
| 2 | FastAPI 기본 구조 + CORS + GET /health | GET /health → 200 OK |
| 3 | Pydantic 스키마 전체 작성 (schemas/pydantic_schemas.py) | import 에러 없음 |
| 4 | POST /scenario/generate — GPT-4o 호출 + DB 저장 | JSON 응답 + DB sessions 행 확인 |
| 5 | GET /scenario/{session_id} — Debate.jsx 진입 시 시나리오 로드 | 시나리오 + 에이전트 정보 정상 반환 |
| 5.1 | GET /session/{session_id} | 올바른 세션 상태 데이터 반환 |
| 6 | POST /chat/message — 메시지 저장 + 턴 결정 | DB messages 행 저장 확인 |
| 7 | GET /chat/stream SSE + 에이전트 응답 스트리밍 | 브라우저 토큰 수신 확인 |
| 8 | Validator 연동 + score_delta + sessions.total_score 업데이트 | 점수 누적 DB 반영 확인 |
| 9 | PATCH /session/surrender + 항복 감지 로직 | both_surrendered 정상 반환 |
| 10 | POST /report/generate + GET /report/{session_id} | 리포트 JSON 반환 확인 |
| 11 | React Home.jsx — 주제 입력 + 시나리오 표시 | 시나리오 카드 렌더링 |
| 12 | React Debate.jsx — 채팅 UI + SSE 연결 + 점수 패널 | 실시간 에이전트 응답 표시 |
| 13 | React Report.jsx — 리포트 UI | 점수 + 피드백 + 탐지 오류 표시 |

**OutThinkAI · Claude Code 통합 개발 지시서 · MVP**